



Métodos Numéricos Computacionais

Igor dos Reis Gomes

RELATÓRIO TRABALHO 4 – INTERPOLAÇÃO DE FUNÇÕES

Métodos Numéricos Computacionais

Bauru

2025

Bauru
2025
Igor dos Reis Gomes

RELATÓRIO TRABALHO 4 – INTERPOLAÇÃO DE FUNÇÕES

Métodos Numéricos Computacionais

Trabalho teórico-prático apresentado como parte da composição da nota da disciplina de Métodos Numéricos Computacionais – Prof. Douglas Rodrigues.

1. OBJETIVO

O objetivo deste trabalho é desenvolver e aplicar os métodos de Interpolação de Lagrange, Interpolação de Newton e Interpolação de Newton-Gregory, utilizando a linguagem Python, para resolver um problema prático fornecido no próprio enunciado do trabalho, envolvendo dados experimentais de temperatura ao longo do tempo. Além disso, busca-se verificar a eficiência e precisão de cada método utilizado.

2. EXPLICAÇÃO DO CÓDIGO

Neste trabalho, foi realizado um programa utilizando a linguagem de programação Python, visando resolver o problema proposto. Assim, o programa foi dividido em 2 arquivos, sendo esses chamados de `funcoes.py` e `igor_dos_reis_gomes_t4.py`.

- No arquivo `funcoes.py`, foram implementadas as funções dos métodos de interpolação (Lagrange, Newton e Newton-Gregory), além de funções auxiliares. Foi utilizada a biblioteca `sympy` para manipulação simbólica de expressões, como a criação de variáveis simbólicas, expansão de polinômios e substituição de valores. Isso permitiu obter os polinômios interpoladores de forma algébrica e calcular seus valores com maior precisão.
- No arquivo `igor_dos_reis_gomes_t4.py`, foram organizadas as resoluções do exercício proposto, além outras funções que tem como objetivo realizar a aplicação dos métodos e comparação de erro e eficiência entre eles.

2.1. Interpolação de Lagrange

```
def interLagrange(pontos_x, pontos_y):  
    itens = len(pontos_x)  
    polinomio = 0  
    x = sp.Symbol('x')  
  
    for i in range(itens):  
        li = 1  
        for j in range(itens):  
            if i is not j:
```

```

        li *= (x - pontos_x[j]) / (pontos_x[i] - pontos_x[j])

    polinomio += pontos_y[i] * li

    return sp.expand(polinomio).evalf(4)

```

Nessa função, temos a aplicação do Método de Interpolação de Lagrange. A função recebe como parâmetros duas listas: os pontos x e seus respectivos valores y. Em cada iteração, é construído o polinômio base $L_i(x)$, que depende da posição do ponto atual em relação aos demais. Cada termo $y_i \cdot L_i(x)$ é somado à variável polinômio para formar o polinômio final. A variável simbólica x é criada para permitir a construção do polinômio de forma simbólica. Ao final, o polinômio é expandido e retornado como resultado.

2.2. Interpolação de Newton

```

def interNewton(pontos_x, pontos_y):

    n = len(pontos_x)

    tabela = []

    polinomio = 0

    x = sp.Symbol('x')

    for _ in range(n):

        linha = [0] * n

        tabela.append(linha)

    for i in range(n):

        tabela[i][0] = pontos_y[i]

    for j in range(1, n):

        for i in range(n - j):

            num = tabela[i + 1][j - 1] - tabela[i][j - 1]

            den = pontos_x[i + j] - pontos_x[i]

            tabela[i][j] = num / den

```

```

polinomio = tabela[0][0]
prod = 1

for i in range(1, n):
    prod *= (x - pontos_x[i - 1])
    polinomio += tabela[0][i] * prod

return sp.expand(polinomio).evalf(4)

```

A função acima é a aplicação da Interpolação de Newton, possui os mesmos parâmetros da função da Interpolação de Lagrange, sendo estes: uma lista de pontos x e outra de pontos y. Primeiramente, é construída uma tabela de diferenças divididas, onde cada linha representa as diferenças sucessivas entre os valores de y sobre os valores de x. Em seguida, o polinômio interpolador é montado utilizando os coeficientes da primeira linha da tabela e os produtos de $(x - x_i)$

2.3. Interpolação de Newton-Gregory

```

def interNewtonGregory(pontos_x, pontos_y, h):
    n = len(pontos_x)
    x = sp.Symbol('x')
    tabela = []
    polinomio = 0

    for _ in range(n):
        linha = [0] * n
        tabela.append(linha)

    for i in range(n):
        tabela[i][0] = pontos_y[i]

    for j in range(1, n):

```

```

        for i in range(n - j):
            tabela[i][j] = tabela[i + 1][j - 1] - tabela[i][j - 1]

s = (x - pontos_x[0]) / h
polinomio = tabela[0][0]
prod = 1

for i in range(1, n):
    prod *= (s - (i - 1))
    polinomio += tabela[0][i] / math.factorial(i) * prod

return sp.expand(polinomio).evalf(4)

```

Agora, por fim, a função apresenta a implementação da Interpolação de Newton-Gregory, que também possui como parâmetros de entrada as listas de x e de y, porém também recebe h, que representa a distância padrão entre os pontos de x. De início, é construída a tabela de diferenças finitas progressivas a partir dos valores de pontos_y, em seguida é calculado a variável s, que será usado para construção do polinômio. O polinômio é então criado somando os termos com as diferenças finitas e os produtos de (s_i) , divididos pelo fatorial correspondente.

2.4. Função de condição da Interpolação de Newton-Gregory

```

def condicaoNewtonGregory(pontos_x):
    n = len(pontos_x)
    h = abs(pontos_x[1] - pontos_x[0])

    for i in range(1, n - 1):
        if abs(pontos_x[i + 1] - pontos_x[i]) != h:
            return False, 0

    return True, h

```

Nessa função, é verificada a condição necessária para aplicar o Método de Newton-Gregory. A função recebe como parâmetro os pontos x e calcula o espaçamento h entre os dois primeiros pontos. Em seguida, percorre os demais pontos e verifica se todos possuem o mesmo espaçamento h . Caso algum intervalo seja diferente desse valor h , a função retorna `false` e zero caso os intervalos não sejam iguais e `true` e o valor de h caso sejam.

2.5. Função de resolução do polinômio

```
def solvePolinomio(polinomio, val, var = 'x'):
    x = sp.Symbol(var)
    res = polinomio.subs(x, val).evalf()

    return res
```

Nessa função, é realizado o cálculo do valor de um polinômio em um ponto específico. A função recebe o próprio polinômio e o valor numérico onde se deseja avaliá-lo. Utilizando a biblioteca `sympy`, a variável simbólica é criada, e o valor é substituído na expressão por meio da função `subs`. Em seguida, o valor calculado é retornado.

2.6. Resolução do exercício

Todas as opções do menu criado, fora as opções 4 e 5, seguem uma estrutura semelhante, que realiza o cálculo de cada interpolador. As listas de pontos estão previamente definidas na função `main`. Logo, segue abaixo o formato de função feito para calcular o polinômio por interpoladores diferentes, sendo neste caso, o Interpolador de Newton.

```
def opcao2(x, y):
    inicio = time.perf_counter()
    polinomio = funcoes.interNewton(x, y)
    tempo = time.perf_counter() - inicio
    res = funcoes.solvePolinomio(polinomio, 2.5)

    print(f"\n[2] Interpolação de Newton:")
```

```

print(f"Polinomio na forma expandida: {polinomio}")

print(f"Valor interpolado em x = 2.5: {res:.4f}")

print(f"Tempo de execução: {tempo:.6f} segundos\n")

return polinomio, res, tempo

```

As funções para a execução das Interpolação de Lagrange, Newton e Newton-Gregory são chamadas com os devidos parâmetros para solucionar o problema, os valores retornados são armazenados em variáveis. Além disso, também é o valor do polinômio quando $x = 2.5$ e o tempo de execução em segundos de cada método.

2.7. Comparação de erro e eficiência dos métodos

```

def opcao5(x, y):

    _, r1, t1 = opcao1(x, y)
    _, r2, t2 = opcao2(x, y)
    _, r3, t3 = opcao3(x, y)

    print("\n[Comparação de Erros]")

    print(f"Erro entre Lagrange e Newton: {abs(r1 - r2):.6f}")
    print(f"Erro entre Lagrange e Newton-Gregory: {abs(r1 - r3):.6f}")
    print(f"Erro entre Newton e Newton-Gregory: {abs(r2 - r3):.6f}")

    print("\n[Comparação de Eficiência]")

    print(f"Tempos: Lagrange: {t1:.6f}s | Newton: {t2:.6f}s |
Newton-Gregory: {t3:.6f}s")

```

Nessa função, é realizada a comparação entre os métodos de interpolação em relação ao erro e suas eficiências. A função recebe os pontos x e y , e executa internamente as funções `opcao1`, `opcao2` e `opcao3`, que aplicam, os métodos de Lagrange, Newton e Newton-Gregory. São obtidos os valores interpolados e os

tempos de execução de cada método. Em seguida, são impressas as diferenças absolutas entre os resultados (erros) e os tempos gastos por cada método, permitindo avaliar qual método foi mais preciso e qual foi mais eficiente.

2.8. Criação dos gráficos

Para a criação dos gráficos, foi realizada a seguinte função:

```
def opcao4(x, y):  
    pol_lagrange = funcoes.interLagrange(x, y)  
    pol_newton = funcoes.interNewton(x, y)  
    _, h = funcoes.condicaoNewtonGregory(x)  
    pol_newtongregory = funcoes.interNewtonGregory(x, y, h)  
  
    polinomios = {  
        'Lagrange': pol_lagrange,  
        'Newton': pol_newton,  
        'Newton-Gregory': pol_newtongregory  
    }  
  
    cores = {  
        'Lagrange': 'red',  
        'Newton': 'green',  
        'Newton-Gregory': 'blue'  
    }  
  
    valores_x = [i * 0.1 for i in range(-10, 61)]  
  
    fig, graficos = plt.subplots(1, 3, figsize=(18, 5))  
  
    for i, (metodo, polinomio) in enumerate(polinomios.items()):  
        valores_y = [polinomio.subs('x', val).evalf() for val in  
valores_x]  
        graficos[i].plot(valores_x, valores_y, color=cores[metodo],  
label=metodo)
```

```

graficos[i].plot(x, y, 'ko', label='Pontos experimentais')
graficos[i].set_title(f'{metodo}')
graficos[i].set_xlabel('Tempo (s)')
graficos[i].set_ylabel('Temperatura (°C)')
graficos[i].grid(True)
graficos[i].legend()

plt.tight_layout()
plt.show()

```

Essa função gera gráficos de comparação entre os métodos de interpolação de Lagrange, Newton e Newton-Gregory. Primeiro, os polinômios interpoladores são obtidos a partir das funções correspondentes. Logo após isso, define-se um intervalo de valores de x para os quais os polinômios serão avaliados e colocados no gráfico. Para cada método, os valores de y correspondentes são calculados e os gráficos são gerados em subplots separados. Cada gráfico mostra a curva do polinômio interpolador e os pontos experimentais originais. Os gráficos são organizados lado a lado e exibidos juntos, já que é melhor para a comparação, pois para todos os três métodos de interpolação os polinômios obtidos são os mesmos.

3. ANÁLISE E DISCUSSÃO

Neste tópico será realizada a comparação entre os métodos de interpolação que foram utilizados, sendo: Lagrange, Newton e Newton-Gregory. Os métodos foram aplicados no mesmo conjunto de dados e produziram o mesmo polinômio expandido, o que mostra que existe uma consistência entre eles. Segue abaixo o conjunto de dados utilizados:

Tabela 1 - Conjunto de dados utilizados

x	0	1	2	3	4	5
f(x)	2,1	3,8	5,2	6,1	7,8	8,9

Além disso, foram analisados o tempo de execução de cada método, o valor interpolado obtido para $x = 2,5$, assim como os erros relativos entre os métodos. A tabela 2 abaixo resume os resultados obtidos:

Tabela 2 - Resultados do exercício

Método	Valor interpolado em $x = 2.5$	Tempo de execução (s)
Lagrange	5,6173	0,046018
Newton	5,6173	0,008020
Newton-Gregory	5,6173	0,004675

Fonte: Elaboração própria

Além da tabela 2, segue os polinômios expandidos para cada método:

- Lagrange: $-0.035x^5 + 0.4125x^4 - 1.633x^3 + 2.388x^2 + 0.5683x + 2.1$
- Newton: $-0.035x^5 + 0.4125x^4 - 1.633x^3 + 2.388x^2 + 0.5683x + 2.1$
- Newton-Gregory: $-0.035x^5 + 0.4125x^4 - 1.633x^3 + 2.388x^2 + 0.5683x + 2.1$

Observa-se que todos os métodos retornam exatamente o mesmo valor interpolado em $x = 2,5$ igual a 5,6173, e o mesmo polinômio expandido, demonstrando a equivalência teórica entre eles.

Agora, a Tabela 3 a seguir mostra os erros absolutos entre os valores interpolados por cada método:

Tabela 3 - Erros absolutos entre métodos

Comparação	Erro absoluto
Lagrange - Newton	0,000000
Lagrange - Newton-Gregory	0,000000
Newton - Newton-Gregory	0,000000

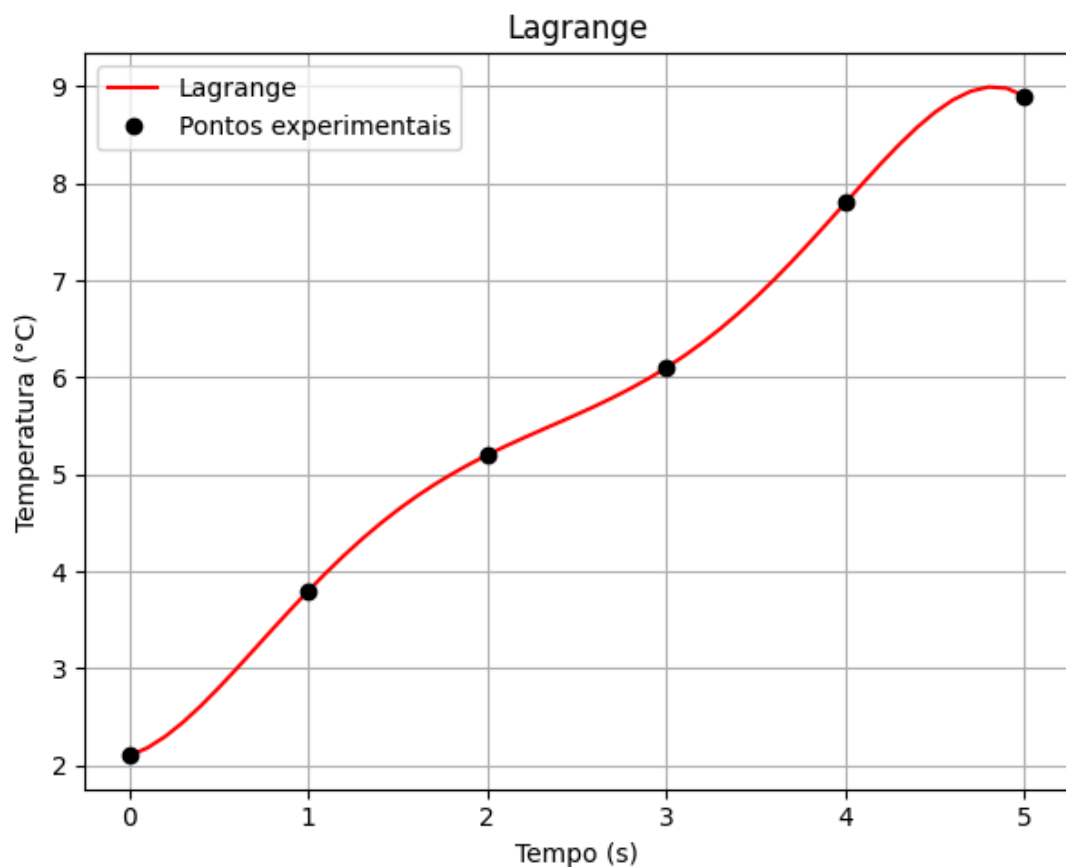
Fonte: Elaboração própria

Com base nesses resultados, conclui-se que os três métodos são igualmente precisos para o conjunto de dados utilizados.

No entanto, ao analisar os tempos de execução, nota-se uma diferença de desempenho entre os métodos. O método de Lagrange foi o menos eficiente, com o tempo de execução muito maior que os outros. O método de Newton teve desempenho intermediário, enquanto o método de Newton-Gregory foi o mais eficiente, apresentando o menor tempo de execução entre os três.

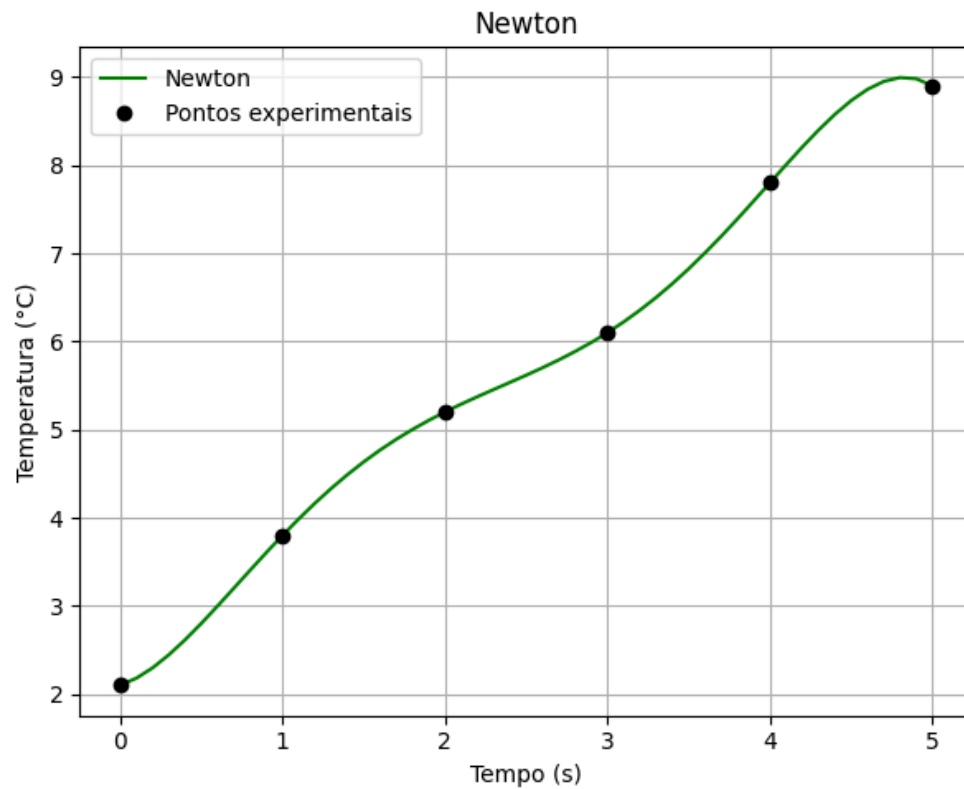
Agora, para final análise, segue os gráficos obtidos para comparação entre os polinômios obtidos

Figura 1 - Gráfico do polinômio utilizando Lagrange



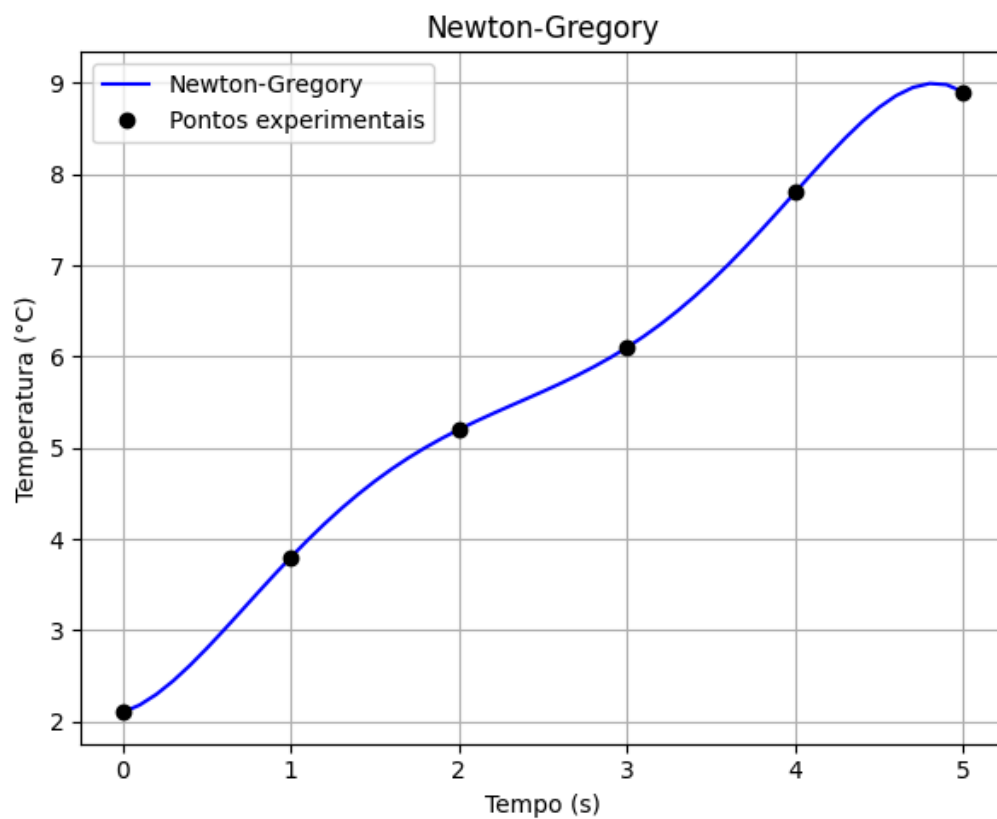
Fonte: Elaboração Própria usando matplotlib

Figura 2 - Gráfico do polinômio utilizando Newton



Fonte: Elaboração Própria usando matplotlib

Figura 3 - Gráfico do polinômio utilizando Newton-Gregory



Fonte: Elaboração Própria usando matplotlib

Observa-se que todos os métodos passam pelos exatos mesmos pontos da tabela, como esperado. Isso confirma que todos os polinômios gerados estão corretos e que todos os métodos funcionaram adequadamente para este caso, já que todos os gráficos tiveram o mesmo comportamento.

4. CONCLUSÃO

Com base na implementação e aplicação dos métodos da Interpolação de Lagrange, Interpolação de Newton e Interpolação de Newton-Gregory, e seus resultados obtidos, é possível observar que todos os três métodos tiveram comportamento idênticos, todos apresentaram o mesmo polinômio expandido e, conseqüentemente, o mesmo valor interpolado em $x = 2,5$, demonstrando assim, que os três métodos são equivalentes entre si. Portanto, conclui-se que todos os métodos são válidos e produzem resultados corretos, porém alguns deles possuem menor custo computacional e melhor estabilidade, e caso utilizados com pontos diferentes dos aplicados neste trabalho, podem ser mais indicados um do que outro. A escolha do método está diretamente ligada com o conjunto de pontos que está sendo disponibilizado.